Employing this technique is simple and works well to guide the LLM to generate replies given its internal knowledge and no other contexts. It's a subtle yet powerful technique that applies the knowledge of the LLM to other applications. This technique, combined with other prompting strategies, is proving effective at replacing other language classification models—models that identify the emotion or sentiment in text, for example.

Open `prompt_flow/zero-shot-prompting/flow.dag.yaml` in the VS Code prompt flow visual editor. This flow is again almost identical to that shown earlier in figure 10.1 but differs slightly in implementation, as shown in listing 10.5.

Listing 10.5 `zero-shot-prompting` flow

```
   Inputs:
        statement  : the statement to be classified
        expected : the expected classification of the statement

    LLM: zero_shot (the prompt used to classify)
        inputs: statement
        outputs: the predicted class given the statement

    Embeddings: uses an LLM embedding model to create the embedding
representation of the text

    Embedding_predicted: embeds the output of the zero_shot LLM
    Embedding_expected: embeds the output of the expected answer

    Python: Evaluation (Python code to measure embedding similarity)
        Inputs:
               Embedding_predicted output
            Embedding_expected output
          Outputs: the similarity score between predicted and expected

   Outputs:
        statement: -> input.statement
        expected: -> input.expected
        predicted: -> output.few_shot
        evaluation_score: output.evaluation

   ### Example Output
{
       "evaluation_score": 1,     #1
       "expected": "neutral",
       "predicted": "neutral",
       "statement": "I think the vacation is okay. "     #2
   }
```

#1 Shows a perfect evaluation score of 1.0
#2 The statement we're asking the LLM to classify